

Today 12:27 PM

I have a VibeCoding lab assignment based on a Donkey Kong Pygame project. I need to use an LLM iteratively to fix the broken code and implement the required features. I'll upload the README/assignment

🔗 https://github.com/SETAPESU26/30_donkeykong

Can you first check the README and tell me what tasks I need to complete? Don't give me all the code at once, let's do it task by task. Before that I have to fork and clone it?

Yep — **fork and clone first**. Since this lab is supposed to show your own iterative work, you want your own GitHub fork rather than editing the original class repo.

I checked the README. It explicitly says there is **1 deliberate bug + 3 unfinished feature functions**, and the submission needs before/after gameplay videos plus the complete LLM chat link. 🔗 [GitHub](#)

Do this first:

1. Open the repo: [SETAPESU26/30_donkeykong](https://github.com/SETAPESU26/30_donkeykong) ↗
2. Click **Fork** → create the fork under your GitHub account.
3. On **your fork**, click **Code** → **HTTPS** and copy the URL.
4. In Terminal, go wherever you keep your lab projects and run:

🔗 Bash

```
git clone YOUR_FORK_URL
cd 30_donkeykong
```

5. Install Pygame if needed:

🔗 Bash

```
pip install pygame
```

6. Run the **original broken version before changing anything**:

```
</> Bash
```

```
python game.py
```

This part matters because the README asks for a **10-second BEFORE video showing the broken behavior**. So don't fix anything yet. [GitHub](#)

The actual tasks we'll do afterward are:

- **Task 1:** Fix the barrel ladder bug — currently roughly **70%** of barrels take a ladder; it should be about **30%**. [GitHub](#)
- **Task 2:** Implement `theme_color(score)` — change the background based on score. [GitHub](#)
- **Task 3:** Implement `on_barrel_jumped(player, barrel)` — add some side effect when you jump over a barrel, such as a floating `+100` . [GitHub](#)
- **Task 4:** Implement `score_multiplier(score)` — return a multiplier such as `2x` after some score threshold. [GitHub](#)

And yeah, we'll **do them one by one**, not dump the whole solution at once.

For now: fork → clone → run the untouched game → record the 10-second BEFORE video.
Then tell me when that's done and we'll start **Task 1 only**.

Ok done

Perfect — now we start **Task 1 only**.

The first task is to fix the **barrel ladder probability bug**.

Right now, barrels are taking ladders way too often — around **70%** of the time — but the assignment wants it to be about **30%**.

Open `game.py` and search for the part where the barrel decides whether to go down a ladder. You'll likely see a condition using `random.random()` or something similar.

Paste just that small barrel/ladder section here, and I'll guide you through exactly what to change and why — no full-code dump.

```
class Barrel:
```

```
def __init__(self):
    self.plat = 4
    self.direction = 1
    self.ladder = None
    self.vy = 0
    self.scored = False
    self.skip = set()
    self.pos = pygame.Vector2(KONG_POS[0] + 40,
PLATFORMS[4][2] - BARREL_R)
```

```
def update(self, dt):
    if self.ladder is not None:
        lx, lower, _ = LADDERS[self.ladder]
        self.pos.y += 90 * dt
        floor = platform_y(PLATFORMS[lower], lx)
        if self.pos.y + BARREL_R >= floor:
            self.plat = lower
            self.ladder = None
        return
    if self.plat is None:
        previous = self.pos.y + BARREL_R
        self.vy += GRAVITY * dt
        self.pos.y += self.vy * dt
        feet = self.pos.y + BARREL_R
        for index, plat in enumerate(PLATFORMS):
            if plat[0] <= self.pos.x <= plat[1]:
                py = platform_y(plat, self.pos.x)
                if previous <= py + 2 and feet >= py:
                    self.plat, self.vy = index, 0
                    break
        return
    plat = PLATFORMS[self.plat]
    slope = plat[3] - plat[2]
    if slope:
        self.direction = 1 if slope > 0 else -1
```

```
self.pos.x += self.direction * BARREL_SPEED * dt
if not plat[0] <= self.pos.x <= plat[1]:
    self.plat, self.vy = None, 0
    return
self.pos.y = platform_y(plat, self.pos.x) - BARREL_R
for index, (lx, _, upper) in enumerate(LADDERS):
    if upper == self.plat and abs(self.pos.x - lx) < 3 and
index not in self.skip:
    self.skip.add(index)
    if random.random() < 0.7:
        self.ladder = index
        self.pos.x = lx
```

Yep — you found the exact bug.

This line is the problem:

Python

```
if random.random() < 0.7:
```

`random.random()` gives a value from `0.0` to `1.0`. So `< 0.7` means the condition is true about **70% of the time**.

For the assignment, change only that line to:

Python

```
if random.random() < 0.3:
```

That makes the barrel take the ladder about **30% of the time**.

So Task 1 fix is literally just:

Python

```
if random.random() < 0.3:
```

Now save and run:

 Bash

```
python game.py
```

Play for a bit and check whether barrels take ladders noticeably less often.

After that, do a Git commit for Task 1:

 Bash

```
git add game.py  
git commit -m "fix barrel ladder probability"
```

Then we can move to **Task 2: `theme_color(score)`** .

Ok do we have to commit after every task is that mentioned explicitly

No bro — **the README does not explicitly say you have to commit after every task.**

I rechecked it. What it explicitly requires is:

- Each task should be done **iteratively using the LLM + your own code review.**  GitHub
- Submission is only:
 - 10-sec **before** video
 - 10-sec **after** video
 - **complete Chat/LLM page link**  GitHub

There is **no instruction saying "commit after each task."**

So you don't need to commit Task 1 right now unless you want the Git history. We can simply continue editing and commit later.

Let's move to Task 2. Find this function in `game.py` :

 Python

```
def theme_color(score):
```

Paste what you have around that function, and we'll do only Task 2.

Today 6:37 PM

```
def theme_color(score):  
    """Return an (r, g, b) background colour for the current  
    score, or None for the default."""  
    pass
```

Yep bro, this is **Task 2**. Right now it does nothing because of `pass` .

The README basically gives us freedom here: `theme_color(score)` should return an **(r, g, b) tuple** based on the player's score, or `None` if you want the normal background.

Let's keep it simple and student-like. For example:

`</>` Python

```
def theme_color(score):  
    """Return an (r, g, b) background colour for the current score, or None for the default  
    if score >= 1000:  
        return (45, 15, 15)  
    elif score >= 500:  
        return (30, 15, 35)  
    return None
```

What this does

Your normal background stays unchanged initially:

`</>` Plain text

score < 500	→ default navy background
500–999	→ dark purple
1000+	→ dark reddish background

The reason we return `None` below 500 is because elsewhere the game has:

`</>` Python

```
screen.fill(theme_color(score) or BG)
```

So if `theme_color(score)` returns `None`, Python uses `BG`, the original background.

This is enough to satisfy the feature without making it unnecessarily complicated.

Add this, run the game, and check that the background changes as your score increases. Then we'll do **Task 3: `on_barrel_jumped()`**.

[ok next](#)

Next is **Task 3: `on_barrel_jumped(player, barrel)`** .

Find this function in `game.py` :

 Python

```
def on_barrel_jumped(player, barrel):
```

and paste that small function here. We'll implement only that feature next.

```
def on_barrel_jumped(player, barrel):  
    """Called when the player clears a barrel; add a bonus effect  
    here."""  
    pass
```

Yep. For **Task 3**, the README allows us to add a bonus effect when Mario successfully jumps over a barrel. A simple choice is a **+100 score bonus**.

But before changing this function, we need to see **how score is stored in your game**, because `score` might be global, part of `player` , or handled somewhere else.

Search for:

 Python

```
score =
```

and also find where this function is called:

 Python

```
on_barrel_jumped(  

```

Paste those small sections here. Then I'll give you the exact Task 3 change without guessing.

 **game(2).py**
Code



this is the file

Yep, I checked the file. The important bit is this:

 Python

```
score += int(100 * (score_multiplier(score) or 1))
on_barrel_jumped(player, barrel)
```

So the **score bonus is already handled outside** `on_barrel_jumped()` .  game

That means for Task 3, we should use `on_barrel_jumped()` for a **visual/side effect**, not add another +100 or you'd double-score.

A simple student-level effect is to make the player do a tiny upward bounce after clearing a barrel:

 Python

```
def on_barrel_jumped(player, barrel):
    """Called when the player clears a barrel; add a bonus effect here."""
    ok next
```

Why this works: `player` already has a vertical velocity `vel.y` , and jumping uses a negative Y velocity, so `-150` gives a small extra hop without being as strong as the normal jump of `-380` .

 game

So replace only the `pass` with:

 Python

```
player.vel.y = -150
```

Then run the game and try jumping over a barrel.

After that we'll do the last one: `score_multiplier(score)` .

ok next

Yep — **Task 4, the last unfinished function: `score_multiplier(score)`** .

Your file currently has:

`</>` Python

```
def score_multiplier(score):  
    """Return a multiplier applied to points earned from clearing a barrel, or None for the  
    pass
```

And when you clear a barrel, the game already does:

`</>` Python

```
score += int(100 * (score_multiplier(score) or 1))
```

So if the function returns `2` , you get **200 points instead of 100**. If it returns `None` , or `1` makes it the normal **100 points**. `</>` game

Let's keep it simple:

`</>` Python

```
def score_multiplier(score):  
    """Return a multiplier applied to points earned from clearing a barrel, or None for the  
    if score >= 500:  
        return 2  
    return None
```

So:

- Score **below 500** → `None` → `1x` → **100 points**
- Score **500 or more** → `2` → `2x` → **200 points**

Replace the `pass` with that.

And with that, we've implemented all **three unfinished functions** plus fixed the barrel probability bug. Next we should run through the game and verify **all four tasks actually work** before you

record the **AFTER** video.

ok done

Nice. Before recording the **AFTER video**, test the finished game once.

Check these four things:

1. **Barrel ladders:** barrels should take ladders only occasionally (~30%), not most of the time.
2. **Theme:** background should change at **100 points**, then again at **300 points**, based on your current `theme_color()` . `</>` game
3. **Barrel jump effect:** when you successfully clear a barrel, you should see/feel the small extra upward bounce.
4. **Multiplier:** before 500 score, a barrel clear gives **100**; at 500+, it gives **200**.

Also test:

- movement + ladders
- lives decrease when hit
- R resets
- reaching the princess still wins

If all of that works, record your **10-second AFTER video** showing the modified game.

One important thing though: since we changed several functions after you uploaded `game.py` , if you want, **upload the final `game.py` now and I can quickly check the whole file for mistakes before you submit.**

`</>` **game(3).py**
Code

Yep bro, I checked the **final** `game.py` . Your four assignment changes are all there correctly:

- **Task 1:** Ladder probability is now `0.3` → ~30%. `</>` game
- **Task 2:** Background changes at score **100** and **300**. `</>` game
- **Task 3:** Clearing a barrel gives the player the little upward bounce with `player.vel.y = -150` . `</>` game
- **Task 4:** At score **500+**, barrel points get a **2x multiplier**. `</>` game

And the multiplier is actually being applied correctly here:

`</>` Python

```
score += int(100 * (score_multiplier(score) or 1))
```

followed by the jump effect. `</>` game

So **no more code changes needed for these tasks**. Your final file looks good.

Now I'd do the submission stuff: run it, record the **10-second AFTER video**, then push your final code to your fork if required by your lab workflow.

Today 8:17 PM

see in just 10 seconds the entire game can't be played... it takes time to win!

Yeah exactly 🤔 **10 seconds is nowhere near enough to finish the whole game.**

The point of the AFTER video is almost certainly **not to show a complete win from start to finish**. It's to prove that your modifications work.

For a 10-second AFTER clip, I'd show the easiest visible change(s):

- Start playing.
- Jump one barrel so the score changes to **100**.
- That immediately shows the **background colour change** too.
- If possible, the little upward bounce from `on_barrel_jumped()` also happens at the same time.
- You do **not** need to reach the princess in those 10 seconds just to prove the feature implementation.

Your score thresholds are actually helpful here: the first theme change happens at **100**, so just **one successful barrel jump** is enough to visibly demonstrate something changed. `</>` game

The **30% ladder probability** and **500-point multiplier** are harder to prove in a 10-second recording because they need multiple barrels/time. I would not artificially alter the code just to make the video easier unless the assignment specifically tells you to demonstrate every feature in the clip.

So I'd treat it like:

BEFORE video: show the original broken gameplay/barrel behavior.

AFTER video: show the modified game running, ideally a barrel jump + score/theme effect.

The code itself is what demonstrates all four implementations; the 10-second video is just quick gameplay evidence.

ya well so I dint show the start of the game I cut a part of the video in bw in which those changes was there as 10s is too less

Yeah, that's completely reasonable. **You don't need to show the game starting from score 0** if the requirement is simply a 10-second AFTER gameplay clip.

Actually, taking a **10-second section from the middle of your gameplay where the implemented changes are visible** is much more useful than wasting half the clip climbing from the beginning 😂.

Just make sure the 10-second AFTER clip visibly shows at least some of the new behavior—ideally the changed background and a barrel jump/score increase. And keep the submitted `game.py` exactly as your final version; don't modify thresholds just for the recording.

So yeah, **your edited 10-second middle section is fine** unless the handout specifically says the recording must begin from game launch.

you check the handout I had sent you

Worked for 5s

I checked the actual handout.

It says:

- **Step 5:** "Record 10 seconds of video before making any changes."
- **Step 8:** "Record another 10 seconds of video again after making all the changes."

 Lab_4_VibeCoding_handout(1)

It does **not** say that either clip has to start from the beginning of the game, and it does **not** say you have to show a complete win in those 10 seconds.

So your approach is fine: for the AFTER video, using a **10-second segment from the middle of gameplay where the implemented changes are visible** satisfies the wording of the handout.

Also, one correction from what I told you earlier: the handout **does** explicitly say to push the code to your repo, and the deliverables are:

- before + after videos
- updated code
- chat history exported as a doc/pdf  Lab_4_VibeCoding_handout(1)

So you're good on the video part.

ya but it doesnt mention to push after each task so I just committed after all

Yep, that's completely fine. The handout only says to **push the code to your repo after making the changes**; it does not require a separate commit or push after each task.

 Lab_4_VibeCoding_handout(1)

So your workflow:

Fork/clone → make Task 1–4 changes iteratively → test → commit everything → push

is consistent with the handout. One final commit containing all the completed changes is fine.